

cow_btrees.py

A step-by-step tour of an SSD-friendly,
copy-on-write B+tree storage engine

Ten lessons, each building on the last, walked with live output from the real engine — not just the source with commentary.

Design & Flow Tutorial

Contents

1 The mental model

The one rule everything else in the file exists to serve.

2 What's physically in the file

Pages, framing, and how a page id becomes a byte offset.

3 Decoding a page

Turning raw bytes back into a LeafNode, field by field.

4 The meta page

Ping-pong slots, CRC32, and why a crash can't corrupt anything.

5 put() — the copy-on-write walk

Tracing one insert through a 3-level tree, level by level.

6 Splitting

Byte-size-aware splits, and the leaf-vs-internal difference.

7 The freelist

Why reclaiming space is self-referential — and the bug that came from it.

8 MVCC

Readers, the watermark, and the race that reclamation exposed.

9 delete()

Underflow, empty-node collapse, and why the bookkeeping needed proof.

10 Group commit

How concurrent writers share one physical commit.

LESSON 1

The mental model

Everything in this file is downstream of one constraint: an SSD can't cheaply overwrite a page in place — it has to erase a whole block first, which is slow and wears the drive out. So the engine makes one foundational rule for itself:

Never modify a page. Only ever write new ones.

That sounds impossible at first — how do you “update” a database if you can never touch what's already there? Here's the trick. The whole database is a tree of pages: root → internal nodes → leaves. Say you want to change one leaf. You can't edit it. So instead:

1. Write a **new** leaf page with the change.
2. Its parent now needs to point at this new leaf instead of the old one — but you can't edit the parent either.
3. So write a **new** parent, pointing at the new leaf.
4. That parent's parent needs updating too... and so on, all the way up to the root.

You end up writing a brand new copy of every node **on the path** from root to the changed leaf.

Everything *off* that path — untouched siblings — doesn't need copying at all; the new tree just points at the same old pages for those parts.

Now here's the part that makes this actually work: once the entire new path is built, exactly **one piece of truly mutable state** is needed — a pointer saying “the current root is page X.” That's the *meta page*. A write is “done” the instant that one pointer flips to the new root. Nothing before that moment is visible to anyone; everything after is.

Why this one idea explains almost everything else in the file

- **Crash safety** falls out for free — the pointer flip either happened or it didn't. There is no half-written state to recover from, so no write-ahead log is needed.
- **MVCC (multiple readers)** falls out for free too — the old root and everything reachable through it are still sitting there, completely untouched. Anyone who grabbed the old root pointer before the flip can keep reading it safely, even while new writes are happening.
- It also **creates** a problem that has to be solved: those old, no-longer-referenced pages are garbage, but their disk space can't be reused immediately — not until it's certain nobody is still reading them. That's the freelist / MVCC-watermark machinery covered in lessons 7 and 8.

One thing worth sitting with: “copy-on-write” here doesn't mean copying the *whole database* on every write — it means copying just the **one path** (root-to-leaf), which for a balanced tree is only a handful of pages, no matter how big the database is.

LESSON 2

What's physically in the file

The file is a flat array of 4096-byte slots. A “page id” is just an index into that array; the only address arithmetic anywhere in the file is:

```
offset = page_id * PAGE_SIZE
```

Two very different kinds of pages live in this array. Dumping a freshly-created database shows why.

```
--- page 0 : a META page (special, fixed format) ---
magic: b'CBT2'  root_id: 2  txn_id: 0  freelist_head: 0  crc32: 3190614525

--- page 1 : the OTHER meta slot -- untouched so far ---
first 4 bytes: all zero

--- page 2 : a real DATA page (the generic frame) ---
length prefix: 4
body: b'\x01\x00\x00\x00'
  body[0] = type byte = 1  (PAGE_LEAF)
  body[1] = reserved  = 0
  body[2:4] = num_keys = 0
```

The meta page's content is always the same five fields — nothing to measure, so no length prefix is needed. Leaf / internal / overflow / freelist pages are the opposite: a leaf with 2 keys and one with 200 keys hold wildly different amounts of real content, but both must fit the same fixed 4096-byte slot. So every data page carries two things a meta page doesn't:

- A **4-byte length prefix** — “the real content ends after N bytes; everything past that is padding, ignore it.”
- A **type byte as the first byte of the body** — so `_read_page` can look at any page id with zero context and immediately know whether it's a leaf, internal, overflow, or freelist page. Pages are *self-describing*.

That page 2 body — type=PAGE_LEAF, 0 keys — is the very first leaf Engine.`__init__` writes when a database is created: the empty root, before `put ( )` has ever been called.

LESSON 3

Decoding a page

Here's a leaf holding two real entries: cat → meow and dog → woof.

```
b'\x01\x00\x00\x02\x00\x03cat\x00\x00\x00\x00\x04meow\x00\x03dog\x00\x00\x00\x00\x04woof'
```

`_decode_leaf` walks this like a tape reader, one field at a time:

offset	bytes	field	value
0	\x01	type	PAGE_LEAF
1	\x00	reserved	-
2:4	\x00\x02	num_keys	2
4:6	\x00\x03	keylen	3
6:9	cat	key	b'cat'
9	\x00	marker	inline
10:14	\x00\x00\x00\x04	vallen	4
14:18	meow	value	b'meow'
18:20	\x00\x03	keylen	3
20:23	dog	key	b'dog'
23	\x00	marker	inline
24:28	\x00\x00\x00\x04	vallen	4
28:32	woof	value	b'woof'

That reconstructs `LeafNode(keys=[cat,dog], values=[meow,woof])` — a lossless round trip. Two design points stand out:

- **Every key and value is length-prefixed.** There's no delimiter character separating fields, because keys/values are arbitrary bytes and could contain any delimiter you might pick. Length-prefixing removes the ambiguity entirely.
- **The marker byte** (offsets 9 and 23, both `\x00` here) works exactly like the page-level type byte, one level down: it tells the decoder whether the next bytes are an inline value (marker=0) or an 8-byte pointer into an overflow chain (marker=1).

Decoding deliberately enforces nothing: no sorting, no validation. It just faithfully deserializes whatever bytes are there. Key order is a promise kept entirely by the *write* side (`_insert_t`, via `_child_index`) — the format is dumb, the tree logic is smart.

LESSON 4

The meta page

Every commit writes to page 0 or page 1, alternating by parity of `txn_id`:

```
slot = txn_id % HEADER_PAGES    # HEADER_PAGES == 2
```

Watching three real commits land:

commit	writes to	txn_id
init	slot 0	0
put(a,1)	slot 1	1
put(b,2)	slot 0	2

Whichever slot a commit doesn't touch is guaranteed to still hold a fully complete, valid previous commit. If commit N crashes partway through writing its slot, the other slot — untouched by N — is sitting there 100% intact. There is never exactly one copy of “the truth” that a crash could catch mid-update.

The CRC: detecting a torn write

```
core = META_MAGIC + struct.pack(">IQI", root_id, txn_id, freelist_head_id)
crc = struct.pack(">I", zlib.crc32(core))
...
if zlib.crc32(core) != stored_crc:
    continue    # torn/corrupt write from a crash -- skip it
```

If a slot's bytes don't hash to its own stored checksum, that slot is discarded entirely — not partially trusted. A live demonstration: a third commit (`put(c, 3)`, `txn 3`) writes to slot 1. A single CRC byte in slot 1 is then flipped, simulating a torn write. Reopening the engine:

```
reopened engine sees: root_id=4 txn_id=2
get(a): b'1'    get(b): b'2'    get(c): None
```

The engine recovered **exactly** to the state right before the corrupted commit — as if `put(c, 3)` never happened at all. Not corrupted, not half-applied: cleanly rolled back to the last good commit. And because tree pages are never modified in place either, the recovered root points to a *complete* tree, not just a valid-looking pointer — the meta page's atomicity is the only atomicity the whole system needs.

LESSON 5

put() — the copy-on-write walk

A real 3-level tree: root (page 21) → internal 18, internal 9 → 14 leaves. Calling `put("k004", "UPDATED")`, where `k004` lives under internal 18, traces through `_insert` like this:

Depth 0 — root (page 21, children [18, 9])

```
node = self._read_page(21)
freed.append(21)                # freed = [21]
i = _child_index(root.keys, 'k004') # -> i=0, go left into child 18
child_new_id, split = self._insert(18, 'k004', 'UPDATED', freed) # recurse
```

Child 9 — the entire other half of the tree — is never read at all.

Depth 1 — internal node (page 18, children [3,5,4,6,8,10])

```
freed.append(18)                # freed = [21, 18]
i = _child_index(node.keys, 'k004') # -> i=1, leaf 5's range
child_new_id, split = self._insert(5, 'k004', 'UPDATED', freed)
```

Depth 2 — leaf (page 5, keys `k004..k008`): bottom of the recursion

```
freed.append(5)                # freed = [21, 18, 5]
values[idx] = 'UPDATED'
new_id = self._alloc_page_id() # -> 19
return 19, None
```

Unwinding back up

```
# depth 1 finishes:
children[1] = 19          # [3,5,4,6,8,10] -> [3,19,4,6,8,10]
return 20, None          # new page id for this level

# depth 0 (root) finishes:
children[0] = 20          # [18,9] -> [20,9]
return 22, None          # new root
```

Exactly what landed on disk: new root 22 with children [20, 9] — 9 bit-for-bit unchanged.
`freed = [21, 18, 5]`: precisely the three pages on the path. The tree held **17 total pages**; updating one key touched exactly **3** — the tree's depth. As the tree grows to thousands of leaves, that ratio only improves, since depth grows logarithmically while total size doesn't.

LESSON 6

Splitting

A node splits based on **encoded byte size**, not key count. `_find_leaf_split / _find_internal_split` try every possible cut point, reject any where either half would still exceed `MAX_BODY_SIZE`, and pick the most evenly balanced one:

```
for i in range(1, n):
    left = header + prefix[i]
    right = header + (total - prefix[i])
    if left <= MAX_BODY_SIZE and right <= MAX_BODY_SIZE:
        diff = abs(left - right)
        if best_diff is None or diff < best_diff:
            best_i, best_diff = i, diff
```

Six uniform ~15-byte entries, artificially small page limit of 80 bytes: the middle cut gives `left = right = 49` — a perfect tie, and exactly what landed on disk:

leaf	keys	body size
4	key0, key1, key2	49 bytes
5	key3, key4, key5	49 bytes

Internal splits are different: the chosen entry is removed, not assigned

An overflowing internal node held 7 separators: [key02, key04, key06, key08, key10, key12, key14] (8 children). The split promoted key08:

```
left = InternalNode(keys[:split_i], children[:split_i + 1])
right = InternalNode(keys[split_i + 1:], children[split_i + 1:])
promoted_key = keys[split_i]      # key08 -- given to NEITHER child
```

```
left child separators: [key02, key04, key06]
right child separators: [key10, key12, key14]
promoted key key08 in left? False    in right? False
```

A separator key holds no data — it's pure routing information (“go right if key $\geq$ this”). Removing it from both children and pushing it up one level loses nothing, because only leaves hold real key/value pairs. This is the classic B-tree “promote the middle key” move, chosen here by byte-balance rather than a fixed midpoint index. Neither function asks “how many keys?” as a stopping condition — only “does the encoded body fit?” With the real 4096-byte page size, a leaf might fit 200+ tiny entries before splitting; fanout is never a constant anywhere in this file.

LESSON 7

The freelist

Freeing a page sounds trivial — remember its id, reuse it later. The catch: that “remembering” has to be durable and mustn't touch pages still in use — and representing “here's a list of free page numbers” itself takes pages. It's self-referential in a way nothing earlier in the file is.

The on-disk structure

Just like an overflow chain, the freelist is a chain of pages, each holding a chunk of free ids plus a pointer to the next chunk:

```
free_pages pool size: 54
the freelist chain itself spans 8 pages: [116, 2, 9, 44, 59, 74, 89, 106]
those pages TOGETHER represent 54 free ids
do the chain's own storage pages appear in their own payload? False
```

The trap

The naive approach — always grow the file for the chain's own storage pages — seems harmless. But representing a bigger pool needs more chain pages, and every one of those is a fresh permanent allocation; meanwhile the previous commit's chain becomes garbage and gets added back into the pool, making it *even bigger* next time. That feedback loop is the real bug this file hit:

4000 inserts → 27,946 file pages, almost entirely freelist chain pages chasing a pool that kept growing because of the freelist's own churn.

The fix

```
storage_ids = [
    self.free_pages.pop() if self.free_pages else self._new_freelist_page_id()
    for _ in range(n_chunks)
]
storage_set = set(storage_ids)
payload_ids = [pid for pid in payload_ids if pid not in storage_set]
```

Source the (small number of) pages needed to *host* the chain from the pool itself, first — then persist whatever's left over as payload, explicitly excluding the ids just repurposed as storage. No double-booking. And it's self-healing: this commit's chain becomes garbage itself next commit, so the pages spent hosting it come right back.

After the fix: **4000 inserts → 40 file pages total**, with no per-commit growth spikes.

LESSON 8

MVCC

A freed page doesn't go straight into the reusable pool — it sits in `pending_free` first, tagged with the transaction that freed it. A live sequence makes the reason obvious:

```
reader opens, pinned at txn_id=20
... 10 more commits happen (txn 21 through 30) ...
after 10 more writes, reader still open:
    free_pages=0, pending_free=11
reader closes
after one more commit:
    free_pages=11, pending_free=0
```

Ten commits happened while that one reader stayed open. Every page any of them freed — eleven total — stayed stuck in `pending_free` the whole time, purely because the reader was still pinned at txn 20. The instant it closed, the very next commit swept all eleven into the reusable pool at once.

The rule

```
watermark = min(self._active_readers) if self._active_readers else None
for pid, freed_at in self.pending_free:
    if watermark is None or watermark >= freed_at:
        self.free_pages.append(pid)
    else:
        still_pending.append((pid, freed_at))
```

A page tagged `freed_at=T` was part of the tree at every txn before `T`. It's safe to reuse only once the *oldest* active reader is already past `T` — i.e. `watermark ≥ T`.

The race this exposed

The watermark check used to run separately from publishing the new root: decide what's safe, *then*, much later, update `root_id/txn_id`. That leaves a gap — a new Reader could register in it, still seeing the old root but *not* counted in the watermark decision that already ran. A page it still needed could get marked reusable and overwritten before it was ever read. This was a real bug: it made the reader/writer stress test fail almost every run once lesson 7's fix let pages get reused fast enough for the race to actually manifest.

```
with self._reader_lock:
    watermark = min(self._active_readers) if self._active_readers else None
    ... decide what's safe ...
    self.root_id, self.txn_id = root, new_txn_id      # published in the SAME lock
```

Since both the watermark decision and registering a new reader require the same lock, the dangerous gap simply doesn't exist anymore: a reader registering just before this section is correctly counted; one registering just after sees the new root already published and has no claim on the old pages at all.

LESSON 9

delete()

Starting 3 levels deep: root 12 → internal 10 (children [4,5]) and internal 11. Deleting both keys from leaf 4 walks through underflow handling step by step.

Step 1 — delete k00: an ordinary leaf update

Leaf 4 still has one key left after this; nothing structurally interesting yet.

Step 2 — delete k01: leaf becomes empty

```
del keys[idx]; del values[idx]
if not keys:
    return None, True, True    # (new_page_id=None, changed=True, became_empty=True)
```

One level up, the now-empty child and its adjacent separator both get dropped:

```
if child_empty:
    del children[i]            # drop the empty child
    if i < len(keys):
        del keys[i]           # drop one adjacent separator too
    else:
        del keys[i - 1]
```

Step 3 — the collapse

```
if len(children) == 1:
    return children[0], True, False    # promote the sole survivor directly
```

One child, zero keys left — this internal node is now a redundant pass-through. No page is written for this level at all; it simply disappears:

```
before: root(12) -> internal(10)[children 4,5] -> leaves
after:  root(12) -> leaf 5 directly -- the middle level is gone
```

Leaf 5 now sits directly under the root. Tree height shrank by one level, in a single delete. Correctness, checked directly:

```
get(k02): b'v2'    <- still finds it, despite the collapsed level
get(k00): None     <- correctly gone
```

Dropping `keys[i]` rather than `keys[i-1]` when a middle child empties out isn't obviously "the right one" — but it doesn't need to be. The removed child held zero keys, so neither surviving neighbor's own key range is affected by which adjacent separator survives. Every real key still resolves correctly via the same " $\text{key} \geq \text{keys}[i] \rightarrow \text{go right}$ " rule from lesson 3 — the surviving separators just end up slightly looser bounds, never wrong ones. What this deliberately does *not* do: merely sparse (not empty) nodes are never merged with a sibling. Always correct, not always space-optimal under heavy delete traffic.

LESSON 10

Group commit

Eight threads, 200 puts each, run concurrently against the same engine:

```
total put() calls made across 8 threads: 1600
physical commits (fsync + meta writes) that actually happened: 600
-> each physical commit absorbed an average of 2.7 concurrent puts
missing/wrong values: 0
```

_submit: queue, then race to become leader

```
op = {... "done": threading.Event(), ...}
with self._batch_lock:
    self._pending_ops.append(op)
    am_leader = self._commit_lock.acquire(blocking=False)
if am_leader:
    try: self._run_commit_batch()
    finally: self._commit_lock.release()
else:
    op["done"].wait()
```

Every call appends an op to a shared queue, then makes one non-blocking attempt to grab `_commit_lock`. Exactly one thread can win at any instant; everyone else falls through to `wait()`, sleeping until the winner signals completion — for their operation too, not just its own.

_run_commit_batch: drain until empty, not just once

```
while True:
    with self._batch_lock:
        batch, self._pending_ops = self._pending_ops, []
    if not batch:
        return
    try: self._commit(batch)
    except Exception as e:
        for op in batch: op["error"] = e
    finally:
        for op in batch: op["done"].set()
```

This grabs the entire current queue in one atomic swap, commits it as a single batch — one fsync and one meta write for the whole batch, the actual group-commit payoff — then loops back. That loop matters: while the leader was committing, other threads could have queued more ops with nobody assigned to process them. Looping until the queue is provably empty is what guarantees every follower eventually gets served.

The expensive part of a commit isn't tree traversal, it's `os.fsync` — a genuine round-trip to stable storage. Without batching, 1600 concurrent puts would mean 1600 separate fsyncs. Under contention, whichever writes overlap while a commit is in flight get folded into that same fsync for free — which is exactly why the average landed at 2.7: while the leader is blocked inside the fsync syscall (which releases the GIL), other threads get a chance to queue up before the next round starts. If a batch commit raises partway through, the whole batch fails together — every op gets the same error, nothing partial ever reaches meta.

How the ten lessons chain together

Lesson 1 set the rule: never modify a page. **Lessons 2–3** showed what a page actually looks like on disk and how it's decoded. **Lesson 4** showed how one small page (meta) makes the whole system's atomicity work. **Lessons 5–6** walked the copy-on-write write path itself, including what happens when a node outgrows a page. **Lesson 7** showed why reclaiming that garbage is its own hard problem, and the bug that came from underestimating it. **Lesson 8** showed why reclamation has to be gated by who's still reading — and the race that surfaced once reclamation got fast enough to matter. **Lesson 9** showed deletion undoing what insertion built, carefully enough to stay correct even while shrinking the tree. And **lesson 10** showed how all of it scales to multiple concurrent writers without giving up any guarantee the first nine lessons established.

Every code excerpt and every numbered result in this document was captured by running the real engine, not written from memory or intended as illustrative pseudocode.